Q25. Application: Fraud Detection System

Question: A fraud detection algorithm follows the recurrence $T(n) = 2T(n/2) + n^2 \log n$. Apply the Master Theorem to determine the time complexity. Compare f(n) with $n^{\log_b a}$ and justify the correct case. Analyze performance for large transaction datasets. Interpret efficiency clearly.

1. Master Theorem Form

The standard Master Theorem form is **$T(n) = aT(n/b) + f(n)$**.

Parameter	Value	Meaning
a	2	Number of recursive subproblems
b	2	Factor by which problem size is reduced
f(n)	$n^2 \log n$	Additional non-recursive work

2. Compare f(n) with $n^{\log_b a}$

Calculate: $n^{\log_2 2} = n^1 = \mathbf{n}$.

Here, $f(n) = n^2 \log n$. Therefore, $n^2 \log n$ is polynomially larger than n. In particular, $f(n) = \Omega(n^{1+\varepsilon})$ for a suitable constant $\varepsilon > 0$.

3. Regularity Condition

For Case 3, the regularity condition is $a\, f(n/b) \leq c\, f(n)$, for some constant $c < 1$. Here:

```
2f(n/2) = 2(n/2)² log(n/2)
        = (n²/2)(log n - 1)
        ≤ c(n² log n)  for sufficiently large n, with c < 1.
```

4. Correct Master Theorem Case

Since f(n) is polynomially larger than $n^{\log_b a}$ and the regularity condition is satisfied, this is **Master Theorem Case 3**.

$T(n) = \Theta(f(n)) = \Theta(n^2 \log n)$

5. Performance for Large Transaction Datasets

The algorithm has quadratic-logarithmic growth. This is substantially slower than O(n) or O(n log n) approaches. If the transaction count is doubled, the dominant work grows by approximately four times (with an additional small logarithmic increase). Therefore, very large fraud-detection datasets can make this approach computationally expensive. A more scalable algorithm or reduction of the n^2 work would improve efficiency.

6. Source Code

```
import math

def fraud_detection_work(n):
    if n <= 1:
        return 1.0
    return 2 * fraud_detection_work(n // 2) + (n ** 2) * math.log2(n)

for n in (2, 4, 8, 16, 32, 64, 128):
    t = fraud_detection_work(n)
    ratio = t / ((n ** 2) * math.log2(n))
    print(f"{n:3d}  {t:10.2f}  {ratio:10.4f}")
```

7. Program Output

```
Q25: Fraud Detection System
Recurrence: T(n) = 2T(n/2) + n^2 log2(n)

 n         T(n)         T(n)/(n^2 log2 n)
-------------------------------------------------
   2          6.00           1.5000
   4         44.00           1.3750
   8        280.00           1.4583
  16       1584.00           1.5469
  32       8288.00           1.6187
  64      41152.00           1.6745
 128     196992.00           1.7176
```

8. Conclusion

Time Complexity: $\Theta(n^2 \log n)$. The recurrence falls under Master Theorem Case 3. Although correct, the method becomes expensive for very large transaction datasets.

Q65. Application: Remote Sensing Image Processor

Question: A satellite image is divided into two equal parts and processed recursively, with additional linear work at each step: T(n) = 2T(n/2) + n. State the Master Theorem T(n) = aT(n/b) + f(n), explain the role of a, b, and f(n), and apply it to determine the time complexity. Also analyze the space complexity of this recursive image processing method.

1. Master Theorem Form and Parameters

Parameter	Value	Role
a	2	Two recursive image-processing subproblems
b	2	Each image portion is half the previous size
f(n)	n	Linear additional work at each recursion level

2. Compare f(n) with $n^{\log_b a}$

$n^{\log■2}$ = n. Since f(n) = n, we have **f(n) = $\Theta(n^{\log_b a})$**.

Therefore, this recurrence satisfies **Master Theorem Case 2**.

3. Apply Case 2

Case 2 gives T(n) = $\Theta(n^{\log_b a} \log n)$. Substituting a = 2 and b = 2:

T(n) = Θ(n log n)

4. Recursion-Tree Interpretation

```
Level 0:         n
Level 1:         2(n/2) = n
Level 2:         4(n/4) = n
...
Number of levels: log2(n)

Total work ≈ n × log2(n) = Θ(n log n)
```

5. Space Complexity

The recursion continues until the image portion reaches size 1. Thus, the maximum recursion depth is log■n + 1, giving **O(log n)** auxiliary stack space when image portions are processed in place or by references/views.

The image data itself requires O(n) storage. Therefore: **auxiliary recursion space = O(log n)**; **including the image storage = O(n)**. If each recursive call copies image data into a new array, the memory usage can be larger.

6. Source Code

```
def image_processing_work(n):
    if n <= 1:
        return 1
    return 2 * image_processing_work(n // 2) + n

def recursion_depth(n):
    if n <= 1:
        return 1
    return 1 + recursion_depth(n // 2)

for n in (2, 4, 8, 16, 32, 64, 128):
    print(n, image_processing_work(n), recursion_depth(n))
```

7. Program Output

```
Q65: Remote Sensing Image Processor
Recurrence: T(n) = 2T(n/2) + n

  n       T(n)        Recursion depth
-------------------------------------------
  2         4               2
  4        12               3
  8        32               4
 16        80               5
 32       192               6
 64       448               7
128      1024               8
```

8. Conclusion

Time Complexity: Θ**(n log n)**. The recurrence is Master Theorem Case 2. Under the in-place processing assumption, the recursive auxiliary space is **O(log n)**, making the approach reasonably scalable for large images.